



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-7 «Программное обеспечение ЭВМ и информационные технологии»

Отчет
по лабораторной работе № 5

Дисциплина: Функциональное и логическое программирование

Студент группы ИУ7-63Б

(Подпись, дата)

Паламарчук А.Н.

(Фамилия И.О.)

Преподаватель

(Подпись, дата)

Толпинская Н. Б.

(Фамилия И.О.)

Преподаватель

(Подпись, дата)

Строганов Ю.В.

(Фамилия И.О.)

2025 г.

Задание 1

Напишите функцию, которая уменьшает на 10 все числа из списка аргумента этой функции, проходя по верхнему уровню списковых ячеек. (* Список смешанный структурированный)

```
(defun sub10_num (lst)
  (mapcar #'(lambda (x) (- x 10)) lst))

(print (sub10_num '(10 20 30))) ;→ (0 10 20)
(print (sub10_num '(5 15 25))) ;→ (-5 5 15)

(defun sub10 (lst)
  (mapcar #'(lambda (x)
    (cond
      ((numberp x) (- x 10))
      (T x)))
    lst))

(print (sub10 '(10 20 a 30))) ;→ (0 10 a 20)
(print (sub10 '(100 x y))) ;→ (90 x y)
```

Задание 2

Написать функцию, которая получает как аргумент список чисел, а возвращает список квадратов этих чисел в том же порядке.

```
(defun square-list1 (lst)
  (mapcar #'(lambda (x) (* x x)) lst))

(print (square-list1 '(1 2 3 4))) ;→ (1 4 9 16)
(print (square-list1 '(-1 -2 -3))) ;→ (1 4 9)

(defun square-list2 (lst)
  (mapcar #'(lambda (x)
    (cond
      ((numberp x) (* x x))
      (T x)))
    lst))
```

```
(print (square-list2 '(1 2 a 4))) ;→ (1 4 a 16)
(print (square-list2 '(0 b 10))) ;→ (0 b 100)
```

Задание 3

Написать функцию, которая умножает на заданное число-аргумент все числа из заданного списка-аргумента, когда

- a) все элементы списка — числа,
- b) элементы списка — любые объекты.

```
(defun mul-num1 (lst num)
  (mapcar #'(lambda (x) (* x num)) lst))

(print (mul-num1 '(1 2 3 4) 2)) ;→ (2 4 6 8)
(print (mul-num1 '(5 10 15) 3)) ;→ (15 30 45)

(defun mul-num2 (lst num)
  (mapcar #'(lambda (x)
    (cond
      ((numberp x) (* x num))
      (T x)))
    lst))

(print (mul-num2 '(1 2 a 4) 2)) ;→ (2 4 a 8)
(print (mul-num2 '(5 b 15) 3)) ;→ (15 b 45)
```

Задание 4

Написать функцию, которая по своему списку-аргументу lst определяет является ли он палиндромом (то есть равны ли lst и (reverse lst)), для одноуровневого смешанного списка.

```
(defun is-palindrome ( lst )
  (reduce #'(lambda (a b) (and a b))
    (mapcar #'eq1 lst (reverse lst))
  )
)

(print (is-palindrome '(1 2 3 2 1))) ;→ T
```

```
(print (is-palindrome '(a b b a))) ;→ T
(print (is-palindrome '(1 2 3))) ;→ NIL
```

Задание 5

Используя функционалы, написать предикат `set-equal`, который возвращает `t`, если два его множества-аргумента (одноуровневые списки) содержат одни и те же элементы, порядок которых не имеет значения.

```
(defun apply-and (lst)
  (reduce #'(lambda (a b) (and a b)) lst))

(defun apply-or (lst)
  (reduce #'(lambda (a b) (or a b)) lst))

(defun set-equal (lst1 lst2)
  (cond
    ((not (eq (length lst1) (length lst2))) nil)
    (T
     (apply-and
      (mapcar #'(lambda (el1)
                    (apply-or
                     (mapcar #'(lambda (el2)
                                   (eq el1 el2))
                                   lst2)))
                    lst1))))))

(print (set-equal '(1 2 3) '(3 2 1))) ;→ T
(print (set-equal '(a b c) '(c b a))) ;→ T
(print (set-equal '(1 2 3) '(1 2 4))) ;→ NIL
```

Задание 6

Написать функцию, `select-between`, которая из списка-аргумента, содержащего только числа, выбирает только те, которые расположены между двумя указанными числами — границами-аргумента и возвращает их в виде списка (упорядоченного по возрастанию (+2балла)).

```
(defun select-between (a b lst)
```

```

(remove-if #'(lambda (el)
              (cond ((< a b) (or (< el a) (> el b)))
                    (T (or (< el b) (> el a)))))
lst))

(print (select-between 6 3 '(1 2 3 4 5 6 7 8 9 10))) ;→ (3 4 5 6)
(print (select-between 3 6 '(10 9 8 7 6 5 4 3 2 1))) ;→ (6 5 4 3)
(print (select-between 30 60 '(1 2 3 4 5 6 7 8 9 10))) ;→ NIL

(defun select-between-sorted (a b lst)
  (sort (remove-if #'(lambda (el)
                      (cond ((< a b) (or (< el a) (> el b)))
                            (T (or (< el b) (> el a)))))
        lst)
    #'<))

(print (select-between-sorted 6 3 '(1 2 3 4 5 6 7 8 9 10))) ;→ (3 4 5
6)
(print (select-between-sorted 3 6 '(10 9 8 7 6 5 4 3 2 1))) ;→ (3 4 5
6)
(print (select-between-sorted 30 60 '(1 2 3 4 5 6 7 8 9 10))) ;→ NIL

```

Задание 7

Написать функцию, вычисляющую декартово произведение двух своих списков-аргументов. (Напомним, что $A \times B$ это множества всевозможных пар (a, b) , где a принадлежит A , b принадлежит B).

```

(defun cartesian-product (lst1 lst2)
  (mapcan #'(lambda (x)
              (mapcar #'(lambda (y) (list x y))
                    lst2))
    lst1
  )
)

(print (cartesian-product '(1 2) '(a b))) ;→ ((1 a) (1 b) (2 a) (2 b))

```

```
(print (cartesian-product '(x y) '(1 2))) ;→ ((x 1) (x 2) (y 1) (y 2))
```

Задание 8

Почему так реализовано reduce, в чем причина?

(reduce #' + ()) -> 0

(reduce #' * ()) -> 1

Функция reduce реализована так для того, чтобы сохранить математическую корректность. Для операции сложения нейтральным элементом является 0, потому что $0 + x = x$ для любого числа x . Значит если список пуст, то результат сложения всех элементов должен быть 0. Для операции умножения нейтральным элементом является 1, потому что $1 * x = x$ для любого числа x . Значит если список пуст, результат умножения всех элементов должен быть 1. Функция reduce возвращает нейтральный элемент для каждой из операции, для сложения это 0, а для умножения 1.

Задание 9

* Пусть list-of-list список, состоящий из списков. Написать функцию, которая вычисляет сумму длин всех элементов list-of-list (количество атомов), т.е. например для аргумента ((1 2) (3 4)) -> 4.

```
(defun sum-lengths (lst)
  (apply #' + (mapcar #' length lst)))

(print (sum-lengths '((a b c) (d e)))) ;→ 5
(print (sum-lengths '(() (1 2 3)))) ;→ 3
```